Hybrid Ant Colony and Q-Learning Algorithm for Swarm Robots: Path Planning and Collision Avoidance in Unknown Environments

1st Andrei DUTCEAC
Department of Electronic Systems and Military Equipment
MTA
Bucharest, Romania
andrei.dutceac@mta.ro

2nd Iulian-Constantin VIZITIU
AOSR
MTA
Bucharest, Romania
iulian.vizitiu@gmail.com

3rd Cristian MOLDER
CERAS
MTA
Bucharest, Romania
cristian.molder@mta.ro

2025 17th International Conference on Electronics, Computers and Artificial Intelligence (ECAI) | 979-8-3315-3352-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ECAI65401.2025.11095526

***Abstract*— This paper presents a hybrid approach combining Ant Colony Optimization (ACO) and Q-learning for swarm robot path planning and collision avoidance in unknown environments. Traditional ACO-based path planning methods face challenges such as local minima and suboptimal solutions when navigating into complex environments. By integrating Q-learning, the proposed method enhances adaptive decision-making, enabling robots to dynamically explore and optimize their routes. The epsilon-greedy strategy balances exploration and exploitation, preventing robots from getting stuck in dead zones and allowing for the discovery of more efficient paths. Simulations conducted in MATLAB demonstrate that the ACO-Q-learning algorithm generates multiple alternative paths, improves overall path efficiency, and enhances swarm coordination compared to standard ACO and Artificial Potential Field (APF) methods. The results highlight the algorithm's potential for real-world swarm robotics applications, where decentralized robots must autonomously navigate and adapt to changing environments.**

Keywords*— *Swarm robotics, Ant Colony Optimization, Q-learning, path planning, reinforcement learning, unknown environments.

I. Introduction

Swarm robotics is a continuously researched domain because of its applications in different fields like agriculture, security and surveillance, area exploration, target search and rescue [1]. Research platforms like The GUARDIANS [2] and SMAVNET Project [3] were developed and tested for search and rescue operations because there are many situations where the environmental conditions could pose a threat to human life, but the swarm can be deployed to complete the mission with minimal losses. What makes swarm robots special compared to a single robot or a coordinated multi-robot team is the decentralized control where a swarm operates without a central controller, each robot follows simple local rules and interacts with its neighbors, leading to collective intelligence. In a disaster scenario, the swarm could explore the zone to search for victims and map the danger area. As an example, in simultaneous localization and mapping (SLAM) for swarm robots is a critical capability that allows a group of decentralized robots to collaboratively map an unknown environment while simultaneously localizing themselves within it [4, 5]. In [6] the authors developed a swarm intelligence-based model for a decentralized drone swarm to detect and extinguish multiple wildfire spots efficiently. They simulated complex scenarios using swarm-intelligence algorithms, incorporating factors such as collision avoidance, extinguishing fluid capacity, and flight range. In [7], the authors proposed a decentralized framework for navigation in dynamic environments, where each robot builds its own local map and shares its planned path with the others. The paths are generated using kinodynamic A* to account for dynamic constraints. Another approach used in [8] presents a simpler navigation strategy in which each robot operates in two modes: Mode 1, moving directly toward the target, and Mode 2, navigating along the edge of the potential field created by stuck robots to bypass them. This approach relies on deploying a large number of robots to ensure successful goal-reaching despite individual failures.

To simulate this complex scenario, we must first study swarm behavior. This can be analyzed from multiple perspectives and for instance, pattern formation is crucial when designing the swarm's flight model. On the other hand, if the focus is on navigation, key aspects such as collective exploration, localization, coordinated motion, and collective transport [9], especially in the context of carrying water to extinguish a fire, must be programmed. Additionally, since robots must make decisions to complete their tasks efficiently, it is essential to simulate task allocation and swarm synchronization to ensure effective coordination.

Swarm behavior in robotics and artificial intelligence is designed by drawing inspiration from natural systems observed in biological communities. These systems include ant colonies, bee swarms, bird flocks, fish schools, and even bacterial colonies, where simple individuals interact through local rules, leading to complex, self-organizing behaviors.

This article proposes a path-planning approach for swarm robots that integrates Ant Colony Optimization (ACO) and Reinforcement Learning (RL) to enhance navigation efficiency by reducing the total distance traveled and addressing the local minima problem.

II. Problem Definition

Most emergencies occur in unknown or dynamic environments requiring swarm robots to navigate efficiently toward a specific target. To achieve this, they must map the area, identify the fastest route, and avoid becoming trapped in dead zones. Dorigo [10] introduced Ant Colony Optimization (ACO), a method inspired by ant colonies, where robots in swarm simulations deposit and detect virtual pheromones to collaboratively identify the most efficient paths. Zhao and Yi [11] integrated the Ant Colony Optimization (ACO) algorithm with the Artificial Potential

979-8-3315-3352-6/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:03:37 UTC from IEEE Xplore. Restrictions apply.

Field (APF) method, where the target acts as an attractive potential, while obstacles generate repulsive potentials. This approach guides the robot toward the target while avoiding obstacles, effectively reducing path length and mitigating the local minima problem.

An enhanced version of the algorithm ACO [12] introduced a new heuristic function, an improved pheromone update rule, and a dynamic pheromone volatilization factor**.** Simulations demonstrated its efficiency in finding the optimal path while effectively avoiding both static and dynamic obstacles. A different approach was proposed in [13], where reinforcement learning was applied to optimize the path for a drone to reach its target. Simulations demonstrated that the Q-Learning algorithm outperformed traditional methods such as Genetic Algorithm (GA) [14], Ant Colony Optimization (ACO), and Particle Swarm Optimization (PSO) [15]**.** However, this approach was designed for a single drone, whereas swarm robotics introduces additional complexity.

III. Proposed Method

A. Ant Colony Algorithm

In this algorithm, each ant represents a robot. Just as ants leave behind a pheromone trail, the robot will deposit a virtual pheromone trail, gradually forming a map. This allows the rest of the group to prioritize paths with higher pheromone concentration, increasing the likelihood of selecting the most frequently traveled route. The algorithm is structured around graphs, where each node represents a potential next point in the path selection process. The probability of a robot moving from point *i* to point *j* is given by:

$$P_{ij}(t) = \frac{[\tau_{ij}(t)]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{s\in allowed}[\tau_{is}(t)]^{\alpha}[\eta_{is}]^{\beta}} \quad (1)$$

where $P_{ij}(t)$ is the pheromone trail, a virtual marker left on the path, influencing the probability of future ants choosing it, η_{ij} is heuristic information (e.g. inverse distance) guiding ants toward promising solutions, α, β control the influence of pheromone vs heuristic information and s∈ allowed are the free points that ant can choose next. After an ant completes its path, pheromones are updated using this rule:

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}(t) \quad (2)$$

where ρ represents the pheromone evaporation factor, with values ranging between (0,1), while (1−ρ) denotes the pheromone retention factor. Additionally, τ_{ij}^{k} represents the pheromone deposit left by the k-th ant at time *t* between points *i* and *j* after its search.

B. Q-Learning Algorithm

Q-Learning is a model-free Reinforcement Learning (RL) algorithm [16] that helps an agent learn optimal actions in an environment through a trial-and-error process. It is based on the Bellman equation and aims to find the best policy for an agent to maximize its cumulative reward over time. The agent, in our case the robot interacts with the environment, making decisions at each step. The environment provides feedback (rewards) based on the agent's actions. There are a few key concepts:

- *state* (*S*) represents the agent's current situation in the environment.
- *action* (*A*) - The agent can take a set of possible actions at each state
- *reward* (*R*) - After taking an action, the agent receives a reward that indicates how good or bad the action was
- *Q-Table (Q-Values)*, where a table stores the expected reward for taking each action in each state

The Q-value is updated using the Bellman equation:

$$Q(s,a) = Q(s,a) + \alpha[R + \gamma * maxQ(s',a') - Q(s,a)] \quad (3)$$

where $Q(s,a)$ is the current Q-value for state *s* and action *a*, α is the learning rate controling how much new information overrides old knowledge ($0<\alpha\leq 1$), $maxQ(s',a')$ is the maximum Q-value for the next state s' (best future action) and s' is the new state after taking action a.

C. Proposed Algorithm

The proposed algorithm integrates ACO's ability to find the shortest path with the Q-Learning method, enabling robots to learn how to avoid collisions with both obstacles and each other. To reduce the randomness in the robots' chosen actions, we enhance the main ant algorithm with a Q-learning component. However, this component is not applied globally across the entire environment, but selectively in critical local states where multiple robots share the same neighborhood. In defining our grid map for simulations, we assign values as follows: free cells are marked with 0, obstacle cells with 1, and the robot's position with 2. In the Q-learning algorithm, tracking the state of each robot at every moment is essential.

Proposed Algorithm

```
Initialize r_states, q_sa table
Initialize parameters
for i=1 in iterations
    Initialize robots' positions, paths
    while (not end)
        for k=1 in robots
            if robot position = target
                update the pheromone map (2)
            end if
            if the robot state has > 1 robot
                add the state to r_states, q_sa
            end if
            if robot state ∈ r_states
                apply the ε–greedy strategy to
                    select the position
                apply Bellman equation (3)
            else apply ACO equation (1)
            end if
            update ant's path
        end for
    end while
    update the pheromone map
end for
```

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:03:37 UTC from IEEE Xplore. Restrictions apply.

To store the robot's 3x3 neighborhood matrix, the variable *r_states* was introduced. Also, this variable will store only the states where at least two robots occupy the same neighborhood matrix. Thus, this allows the robots to train and learn movement patterns to avoid close encounters. In opposite, when a robot is alone in its state, it will follow the ACO algorithm using (1) to determine its next move.

To initialize the Q-Table, we define the variable *q_sa* to store all states from *r_state*. For each state, eight values are assigned, corresponding to the robot's eight possible movement directions. Each robot that reaches the end will receive an update of the pheromone map using (2). ACO chooses the next position based on pheromone level and heuristic factor η_{ij}. To reduce the chances of getting stuck and increase the probability of reaching the target, this approach was enhanced using the following equation:

$$\eta_{ij} = \frac{d_{sj}}{d_{ij} + d_{i_end}^3} \tag{4}$$

where d_{sj} is the distance from the start point to the current position, d_{ij} is the distance from the current position to the next point, d_{i_end} is the distance from the current point to the target.

During the training process, we use (3) to update the Q-table. For each possible action in a given state, we calculate the probability of selecting that action. Additionally, using (4) enhances the algorithm's ability to choose the next point nearer to the goal by amplifying the impact of the total distance via an increased exponent factor. By following a specific policy, the best action for a state is chosen based on the action with the highest value, which is determined by the previous value and the received reward. The Q-learning method combines exploration and exploitation techniques. By exploration, the robot discovers new states, adding them to the Q-table and computing action-value pairs for each state. Exploitation, on the other hand, selects the action with the highest value for a given state. Over time, as states are revisited, previously calculated values are updated, refining the policy and enhancing decision-making. An effective approach incorporating both techniques is the epsilon-greedy strategy, which ensures a balance between exploration and exploitation. Finally, with a probability of ε, a random action is taken to explore new possibilities, while with a probability of 1− ε, the action with the highest known value is selected.

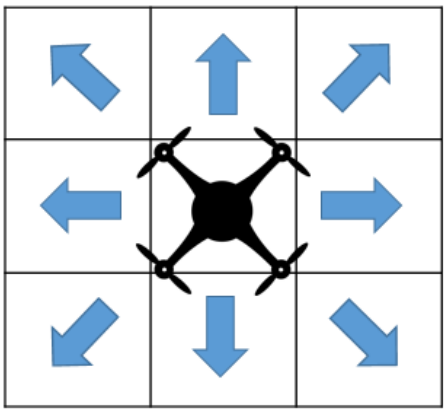


Figure 1. Robot movement directions

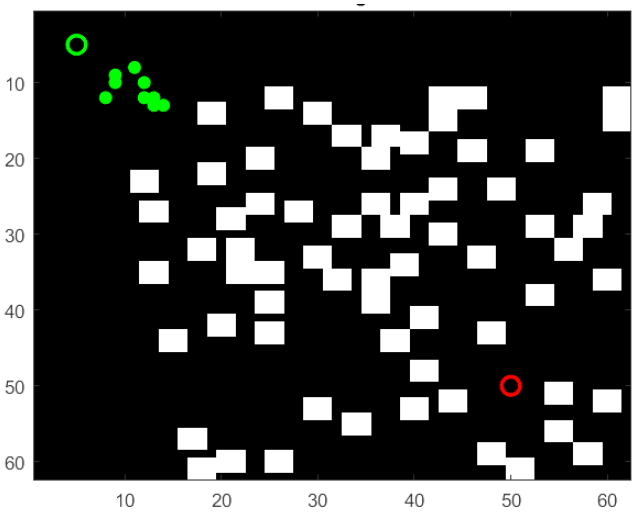


Figure 2. The simulation map

IV. Experimental Results

A. Environment Design

To simulate the tested algorithms in our area of interest, data on the position of each object and the paths of the robots was required. So, to achieve this task, it considers one robot from the swarm as a moving point across the map. In Figure 1, the robot is represented on a grid map along with its possible movement directions. The 3×3 matrix places the robot at the center, displaying its neighbor's positions, which it can sense and navigate accordingly.

The map was designed in MATLAB with a 62×62 grid, where random obstacles are placed, each occupying a 2×2 grid space (see Figure 2). To ensure consistency in map generation and maintain the same format, MATLAB's *seed* function is used, allowing the reproduction of the same map based on a given parameter. Additionally, the difficulty level can be increased by introducing randomly generated obstacles using the same randomization process with a specific seed value. The swarm's initial position is set within a 5×5 grid area, starting at coordinates (5,5), while the target is located at (50,50). The swarm had knowledge only of the target's position, and its mission is to discover the optimal paths, ensuring that all robots reach the target in the shortest possible time.

B. Experimental Results

The simulations were performed using MATLAB R2024b on a Dell laptop equipped with an Intel Core i7-1165G7 processor, 16 GB RAM, and Windows 10. We

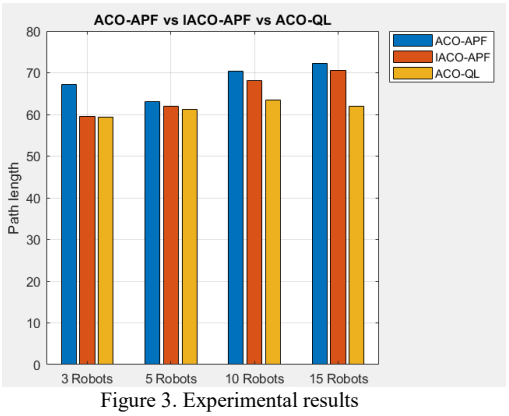


Figure 3. Experimental results

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:03:37 UTC from IEEE Xplore. Restrictions apply.

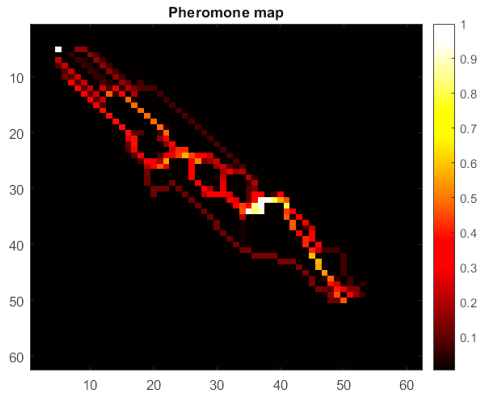


Figure 4. Pheromone map from ACO-APF

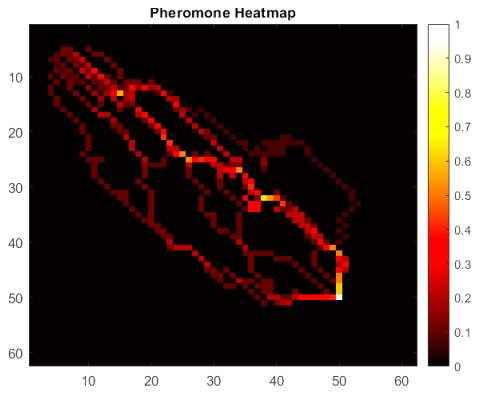


Figure 5. Pheromone map from ACO-QL

compare the proposed algorithm with the ones from [10] and [11], specifically the ant colony combined with the artificial potential field algorithm and its improved version. As the cost function, we considered the average path length of all swarm robots used to evaluate swarm behavior, the number of robots reaching the target, and how the path evolves based on the number of robots and obstacles.

Figure 3 presents the simulation test on 10 different maps with different swarm size from 3 to 15 robots. Robots communicate indirectly through pheromone trails and proximity-based Q-learning state sharing. Each robot is aware of its position and maintains a Q-table that stores state-action values only when at least one other robot is detected in its neighborhood. For instance, in a single simulation, a robot may record a few hundred such states, resulting in a Q-table with n rows (representing the saved states) and m columns (corresponding to the 8 possible actions). For each state-action pair, the algorithm computes a Q-value based on a reward function: -10 if the resulting state is an obstacle or occupied by another robot, and +1 if the state is free.

The proposed algorithm is only tested in this simulation, as the training phase was conducted beforehand. The training involved running several iterations on different maps to learn swarm behavior. For the proposed test simulation, all algorithms were executed for a single iteration. The results indicate an improvement in path length for the ACO-QL algorithm and suggest that there is no fixed rule for achieving a shorter path based on swarm size, as all robots attempt to learn the same navigation rules. As it can be observed, for a swarm of 3 robots, the average path lengths

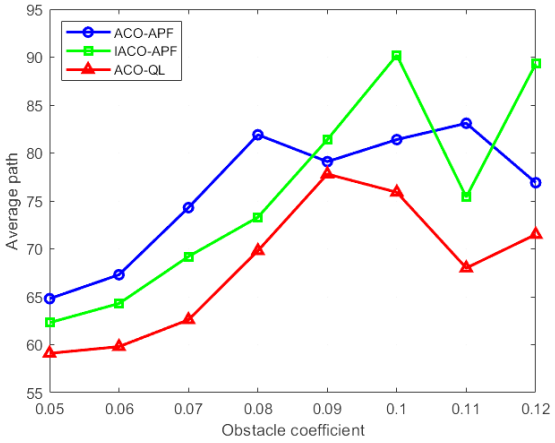


Figure 6. Map difficulty comparison

TABLE I. TRAINING EFFICIENCY

Average path	Training iterations					
	1	*2*	*3*	*6*	*10*	*20*
Map 1	76.8	82.8	69.5	**69.1**	72.7	86.2
Map 2	61.1	62.7	65.9	**59.2**	62.8	63.8
Map 3	67.4	65.8	**63.4**	69.2	64.4	69.2

are 67.2 for ACO-APF, 59.6 for the improved ACO, and 59.4 for the proposed algorithm, indicating only a small difference. However, this difference becomes more significant as the number of robots increases. For instance, with 15 robots, the improved ACO records an average path length of 70.6, while ACO-QL achieves 62.08.

Figures 4 and 5 illustrate the pheromone heatmap generated after the simulation stage, along with the intensity of the detected paths. The achieved results show that ACO-QL provides more alternative paths compared to the potential field method. This contributes to overcoming the local minima problem, where robots may become trapped in dead zones created by obstacles.

In a similar manner, we tested the algorithm by varying the number of obstacles in the environment, gradually increasing the obstacle coefficient from 0.05 to 0.12. Figure 6 illustrates the simulation of a randomly generated map with an increasing number of obstacles and a swarm of 10 robots. The results highlight that ACO-QL tends to find shorter paths. Interestingly, in some cases, the swarm identifies better paths even in maps with more obstacles, depending on their position, which significantly influences the robots' behavior. It is also observed that, in some cases, the behavior of ACO-APF leads to more optimal paths than those produced by the improved ACO.

In general, reinforcement learning yields better results with increased training. However, in our case, the focus is specifically on avoiding collisions and dead zones for the robots. Simulations demonstrate that the algorithm performs effectively with just 6 iterations of training data, as shown in the TABLE I. The relatively low number of required training iterations is due to the compact state space used, so each robot only trains on local 3×3 grid configurations. Q-learning is triggered only when a robot is near another (to avoid collisions), limiting the number of required updates. One

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:03:37 UTC from IEEE Xplore. Restrictions apply.

issue with excessive training is that the robots may start repeating the same movements in loops without ever reaching the target. To prevent this, it's important to retain an element of randomness in their decision-making process.

To assess the computational complexity of the proposed algorithms, we employed MATLAB's built-in profile function. The simulations were conducted on 10 different maps, each with a difficulty coefficient of 0.08, using a swarm of 10 robots. The average simulation times recorded were: 11.12 seconds for ACO-APF, 9.90 seconds for IACO-APF, and 17.47 seconds for ACO-QL. The higher execution time of ACO-QL reflects the additional computational overhead required for evaluating and updating the Q-values corresponding to each state-action pair.

Furthermore, the success rate of each algorithm was evaluated, defined as the percentage of robots successfully reaching the target. When increasing the difficulty coefficient, the following results were observed: for a coefficient of 0.08, all algorithms achieved a 100% success rate; at 0.11, success rates dropped to 86% for ACO-APF, 90% for IACO-APF, and remained at 100% for ACO-QL; for a higher difficulty of 0.14, the success rates decreased significantly: 44% for ACO-APF, 45% for IACO-APF, while ACO-QL maintained 100%. (Figure 7)

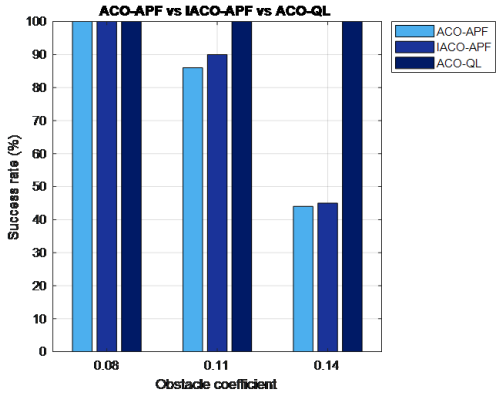


Figure 7. Success rate

These findings suggest that the reinforcement learning component in ACO-QL significantly enhances performance in complex environments by enabling more flexible path planning and facilitating better convergence toward the goal.

V. Conclusion

This paper employs ACO and Q-learning to address path planning and obstacle avoidance for a swarm of robots. Q-learning allows agents to improve their strategies based on past experiences, optimizing paths over time. The epsilon-greedy strategy enables the robot to explore new routes, avoiding deadlocks and reliance on a single optimal path. The proposed algorithm continuously adjusts the navigation policy based on the state evaluation, leading to route diversification and better adaptability to variable environments. To evaluate performance, the area covered by the robots was compared to the total area of the environment (Figures 4–5). The results show that ACO-APF achieved a coverage of 4.92%, IACO-APF reached 7.28%, while ACO-QL achieved 10.25%, indicating a notable improvement. Furthermore, increasing the number of robots reduces the likelihood of dead zones, as each robot tends to follow the pheromone trails left by others. Generating multiple paths is a major advantage in an unknown environment, as it provides alternatives in case a route becomes inaccessible. Since this article focuses solely on grid-based environments, future work will explore its extension to continuous 3D spaces, where the continuous nature of states and actions requires specialized reinforcement learning algorithms such as Actor-Critic or Proximal Policy Optimization.

References

[1] Shahzad, M.M.; Saeed, Z.; Akhtar, A.; Munawar, H.; Yousaf, M.H.; Baloach, N.K.; Hussain, F. A Review of Swarm Robotics in a NutShell. Drones 2023, 7, 269. https://doi.org/10.3390/drones7040269.

[2] Saez-Pons, J.; Alboul, L.; Penders, J.; Nomdedeu, L. Multi-robot team formation control in the GUARDIANS project. Ind. Robot. Int. J. 2010, 37, 372–383.

[3] Hauert, S., Zufferey, J.-C., & Floreano, D. (2008). Evolved swarming without positioning information: an application in aerial communication relay. Autonomous Robots, 26(1), 21–32. doi:10.1007/s10514-008-9104-9.

[4] Malavazi, F. B. P., Guyonneau, R., Fasquel, J.-B., Lagrange, S., & Mercier, F. (2018). LiDAR-only based navigation algorithm for an autonomous agricultural robot. Computers and Electronics in Agriculture, 154, 71–79. doi:10.1016/j.compag.2018.08.034 .

[5] Durrant-Whyte, H., & Bailey, T. (2006). Simultaneous localization and mapping: part I. IEEE Robotics & Automation Magazine, 13(2), 99–110. doi:10.1109/mra.2006.1638022.

[6] I. HUSSEIN, S. SARASWATHY, M. MAHMOUD, A. NAJAH, M. ALKILABI, "Nature-Inspired Drone Swarming for Wildfires Suppression Considering Distributed Fire Spots and Energy Consumption", 2023, 10.1109/ACCESS.2023.3279416.

[7] S. Wu, G. Chen, M. Shi, and J. Alonso-Mora, "Decentralized Multi-Agent Trajectory Planning in Dynamic Environments with Spatiotemporal Occupancy Grid Maps," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan: IEEE, May 2024, pp. 7208–7214. doi: 10.1109/ICRA57147.2024.10610670.

[8] Tsunoda, Y., Otsuka, S., Ito, K., Xiao, R., Naniwa, K., Sueoka, Y., & Osuka, K. (2024). Power in Numbers: Primitive Algorithm for Swarm Robot Navigation in Unknown Environments. *arXiv preprint arXiv:2410.13149*.

[9] Schranz, M., Umlauft, M., Sende, M., & Elmenreich, W. (2020). Swarm Robotic Behaviors and Current Applications. Frontiers in Robotics and AI, 7. doi:10.3389/frobt.2020.00036 .

[10] M.Dorigo, G.DiCaro, and L.M.Gambardella, "Ant algorithms for discrete optimization," Artif. Life, Vol.5, no.2, pp. 137–72, 1999.

[11] Zhao, D., Yi, J. (2006). Robot Planning with Artificial Potential Field Guided Ant Colony Optimization Algorithm. In: Jiao, L., Wang, L., Gao, X., Liu, J., Wu, F. (eds) Advances in Natural Computation. ICNC 2006. Lecture Notes in Computer Science, vol 4222. Springer, Berlin, Heidelberg. https://doi.org/10.1007/11881223_28

[12] Chen, Y., Bai, G., Zhan, Y., Hu, X., & Liu, J. (2021). "Path Planning and Obstacle Avoiding of the USV Based on Improved ACO-APF Hybrid Algorithm With Adaptive Early-Warning". IEEE Access, 9, 40728–40742. doi:10.1109/access.2021.3062375.

[13] E. M. B. Laoula, O. Elfahim, M. Youssfi, and O. Bouattane, "Drone path optimization in complex environment based on Q-learning algorithm," in 2022 International Conference on Intelligent Systems and Computer Vision (ISCV), Fez, Morocco: IEEE, May 2022, pp. 1–7. doi: 10.1109/ISCV54655.2022.9806077.

[14] Goldberg, D.E. Genetic Learning in Optimization, Search and Machine Learning; Addisson Wesley: Boston, CA, USA, 1994.

[15] Poli, R.; Kennedy, J.; Blackwell, T. Particle swarm optimization: An overview. Swarm Intell. 2007, 1, 33–57.

[16] C. J. Watkins and P. Dayan. Q-learning. Machine learning, 8(3-4):279 292, 1992.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:03:37 UTC from IEEE Xplore. Restrictions apply.